TRIBHUVAN UNIVERSITY

FACULTY OF HUMANITIES AND SOCIAL SCIENCE

A PROJECT REPORT

On the Topic

"REAL TIME CHAT APPLICATIOM"

Submitted To:

Department of Computer Application

Whitefield Int'l College

In partial fulfillment of the requirements for Bachelor Degree in Computer Application

Submitted by:

Sophan Shrestha

BCA 6th Semester

Symbol no: 49702038

TU Reg No: 6-2-497-19-2021

Under the Supervision of

Mr. Saroj Singh

Date: 29th June, 2025



Tribhuvan University

Faculty of Humanities and Social Science

Whitefield Int'l College

SUPERVISOR'S RECOMMENDATION

I here by recommend that this project, prepared under our supervision by **Mr. Saroj Singh** and entitled **"REAL-TIME CHAT APPLICATION,"** in partial fulfillment of the requirements for the degree of Bachelor of Computer Application, is recommended for the final evaluation.

.....

Signature

Mr. Saroj Singh

SUPERVISOR

Department of BCA

Whitefield Int'l College

Date: 29th June, 2025

Tribhuvan University

Faculty of Humanities and Social Sciences

Whitefield Int'l College

LETTER OF APPROVAL

This is to certify that the project prepared by **Sophan Shrestha** entitled **"REAL-TIME CHAT APPLICATION,"** in partial fulfillment of the requirements for the degree of Bachelor in Computer Application, has been evaluated. In our opinion, it is satisfactory in both scope and quality as a project for the required degree.

_____ Supervisor **Mr. Saroj Singh** Instructor Whitefield International College Nayabazaar, Kathmandu	_____ HOD/ Coordinator **Er. Sujan Shrestha** Program Coordinator, BCA Whitefield Int'l College Nayabazaar, Kathmandu
_____ **Mr. Saroj Singh** Internal Examiner	_____ **Mr. Roshan Tandukar** External Examiner

ABSTRACT

The Real-Time Chat Application project is developed to provide a seamless, responsive, and secure communication platform that facilitates instant messaging between users. Designed to cater to both individual and group interactions, the application aims to enhance digital collaboration and user connectivity in real time, regardless of geographical boundaries.

Built on the robust and scalable MERN stack — MongoDB, Express.js, React.js, and Node.js the system ensures efficient back-end performance, a dynamic front-end interface, and secure data management. Real-time communication is enabled through WebSockets (using Socket.IO), providing instant message delivery and updates with minimal latency.

The application incorporates a modular architecture, with a focus on scalability and maintainability. User authentication and authorization are handled securely using technologies such as JWT (JSON Web Tokens) and bcrypt for password hashing, ensuring data protection and controlled access.

The intuitive and responsive user interface, developed with React.js and styled using modern CSS frameworks, ensures a smooth and user-friendly experience across all device types. Key features include private and group chats, online/offline status indicators, message delivery/read receipts, and real-time notifications.

This project demonstrates a practical application of modern web development technologies and real-time communication protocols, delivering a reliable and interactive messaging solution. It is a testament to best practices in full-stack development and offers potential for further enhancement into a scalable enterprise-level communication tool.

Keywords: *Real-Time Chat Application, MERN Stack, Secure User Authentication, WebSocket Integration, Private Messaging.*

ACKNOWLEDGEMENT

I am sincerely grateful to everyone who supported and guided me throughout the development of the **Real-Time Chat Application**. This project has been an enriching experience, and I truly appreciate the valuable insights and encouragement provided along the way.

First and foremost, I would like to express my heartfelt gratitude to my supervisor, **Mr Saroj Singh**, for her expert guidance, constructive feedback, and continuous encouragement. Her mentorship has been instrumental in shaping the direction and success of this project.

I am also deeply thankful to my parents and friends for their unwavering support, patience, and motivation throughout the development process. Their belief in my capabilities and their thoughtful suggestions have significantly contributed to the successful completion of this project.

Lastly, I extend my sincere thanks to all those who inspired and supported me during this journey. Your encouragement helped me overcome challenges and stay focused on my objectives while building this real-time communication platform.

Thank you all!

With Respect,

Sophan Shrestha (49702038)

TABLE OF CONTENT

SUPERVISOR'S RECOMMENDATION i
LETTER OF APPROVAL ii
ABSTRACT iii
ACKNOWLEDGEMENT iv
TABLE OF CONTENT v
LIST OF FIGURES vii
LIST OF COLLECTION viii
LIST OF LISTING x
LIST OF ABBREVIATION xi
CHAPTER 1 1
INTRODUCTION 1
1.1 Introduction 1
1.3 Objectives 2
1.4 Scope and Limitation 2
1.4.1 Scope 2
1.4.2 Limitation 2
1.5 Development Methodology 3
1.6 Report Organization 4
CHAPTER 2 4
BACKGROUND STUDY AND LITERATURE REVIEW 5
2.1 Background Study 5
2.2 Literature Review 5
CHAPTER 3 6
SYSTEM ANALYSIS AND DESIGN 7
3.1 System Analysis 7
3.1.1 Requirement Analysis 7
3.1.2 Feasibility Analysis 9
3.1.3 Object Modelling: Class and Object Diagram 11
3.1.4 Dynamic Modelling: State and Sequence diagram 13
3.1.5 Process Modelling: Activity Diagram 15
3.2 System Design 16
3.2.1 Component Diagram 16
3.2.2 Deployment Diagram 17

Web App 17
Application Server 18
Database Server 18
3.2.3 Database Schema Design 18
3.3 Algorithm Used 22
CHAPTER 4 23
IMPLEMENTATION AND TESTING 23
4.1 Implementation 23
4.1.1 Tools Used 23
4.1.2 Implementation Details of Modules 25
4.2 Testing 26
Postman API Testing 27
4.2.1 Testing cases for Unit Testing 28
CHAPTER 5 34
CONCLUSION AND FUTURE RECOMMENDATIONS 34
5.1 Conclusion 34
5.2 Lesson Learnt 34
5.3 Future Recommendation 34

LIST OF FIGURES

Fig 1.1 Agile Methodology……...3

Fig 3.1 Use Case Diagram…….9

Fig 3.2 Project Grant Chart……11

Fig 3.3 Class Diagram……12

Fig 3.4 Object Diagram…….....13

Fig 3.5 State Diagram…….....14

Fig 3.6 Sequence Diagram…….....15

Fig 3.7 Activity Diagram……16

Fig 3.8 Component Diagram……...17

Fig 3.9 Deployment Diagram…….....18

LIST OF COLLECTION

Collection 3.1 User and Admin Collection…….20

Collection 3.2 Messages Collection ……..20

Collection 3.3 Conversation Collection…….…….21

LIST OF TABLES

Table 4.1 Test case for user registration…….....27

Table 4.2 Test case for user email...…….....28

Table 4.3 Test case for user password…….....28

Table 4.4 Test case for Admin register…….....29

Table 4.5 Test case for Admin login…..…….....30

Table 4.6 Test case for search user…….....31

Table 4.7 Test case for chat ……...…….....32

Table 4.8 Test case for edit profile ….…….....32

Table 4.9 Test case to admin user detail ……..…….....33

Table 4.10 Test case to admin delete user…….…….....33

LIST OF Algorithm

Listing 3.1 Code snippet of Binary Search Algorithm…….22

LIST OF ABBREVIATION

API – Application Programming Interface

CRUD – Create, Read, Update, Delete

CSS – Cascading Style Sheets

DOM – Document Object Model

FRs – Functional Requirements

HTML – Hypertext Markup Language

HTTP – Hypertext Transfer Protocol

HTTPS – Hypertext Transfer Protocol Secure

JWT – JSON Web Token

JS – JavaScript

JSON – JavaScript Object Notation

MERN – MongoDB, Express.js, React.js, Node.js

NFRs – Non-Functional Requirements

NoSQL – Non-Structured Query Language

REST – Representational State Transfer

SSL – Secure Sockets Layer

UI – User Interface

UX – User Experience

CHAPTER 1

INTRODUCTION

1.1 Introduction

The **Real-Time Chat Application** is a modern communication platform designed to provide fast, secure, and seamless messaging between users. It enables individuals and groups to engage in real-time conversations, making it ideal for both personal and professional use. Built for efficiency and user convenience, the application ensures low-latency message delivery using WebSocket technology, offering instant communication across devices. The platform features a clean, intuitive interface with organized chat channels, user presence indicators, and message read receipts, helping users stay connected and informed at all times. Security and reliability are central to the system's design, with robust user authentication and encrypted data handling to protect user information and ensure safe interactions. Whether for one-on-one chats or group discussions, the platform delivers a smooth and engaging communication experience.

By streamlining digital interactions through a responsive and user-friendly interface, the Real-Time Chat Application offers a reliable and scalable solution for modern messaging needs.

1.2 Problem Statement

People often face challenges when trying to communicate instantly and efficiently, especially in fast-paced or remote environments. Traditional messaging methods, such as email or delayed response platforms, can hinder collaboration, reduce productivity, and lead to miscommunication—particularly when real-time interaction is essential.

Without a reliable and secure platform for instant communication, users struggle to stay connected, share updates promptly, or coordinate tasks effectively. This becomes even more problematic in team-based settings or during urgent conversations where immediate feedback is crucial.

A real-time chat application that enables instant, secure messaging between users can significantly improve communication. By offering features like live chat, message notifications, and user presence indicators, such a platform simplifies the way individuals and

teams interact, saving time, enhancing clarity, and fostering more dynamic and responsive communication.

1.3 Objectives

The main objectives of the **Real-Time Chat Application** are:

1. To create a safe, fast, and easy-to-use platform for chatting in real time.
2. To make sure messages are delivered quickly and reliably using technologies like WebSocket (e.g., Socket.IO).
3. To add strong login and user access features to keep conversations secure.
4. To design a clean and responsive chat interface that works well on all types of devices.

1.4 Scope and Limitation

1.4.1 Scope

Real-Time Communication: The chat application enables users to send and receive instant messages in real time. It supports one-on-one conversation ensuring smooth and immediate communication between users.

User-Friendly Interface: The system features an intuitive and responsive interface designed for ease of use. Users can quickly navigate between chats, manage contacts, and access conversation history across desktop and mobile devices.

Secure Messaging: The platform implements secure user authentication using modern methods such as JWT. It ensures that messages are transmitted securely and privately, protecting user data and communication.

1.4.2 Limitation

Lack of Face-to-Face Interaction: Unlike in-person conversations, users cannot rely on facial expressions or vocal tone, which may sometimes lead to misunderstandings or lack of emotional context in communication.

Device Compatibility and Performance: Although the application is optimized for various devices, performance may vary depending on the user's hardware, browser, or operating system. Older devices may experience lag or limited functionality during active chat sessions.

1.5 Development Methodology

Methodologies are a set of guiding principles and processes used to plan, manage, and execute projects. The project management methodology you choose determines how work is prioritized and completed.

Agile Methodology:

Our real-time chat application is developed using the Agile methodology, which enables the us to deliver functional features through short, iterative development cycles. By breaking the project into manageable sprints, we can rapidly adapt to changing user needs and continuously enhance the system's performance and usability.

Agile encourages strong collaboration among developers, UI/UX designers, and stakeholders. Regular feedback loops allow us to identify and resolve issues early, ensuring that the application remains user-centered and responsive to evolving communication needs [1]

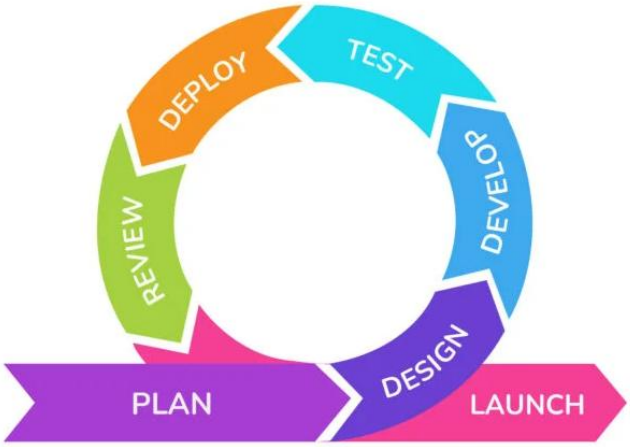


Fig 1.1 Agile Methodolog

In the case of our chat application, the Agile methodology helped us build the system step by step. We started with basic features like user registration and one-on-one messaging, then added more advanced features like file sharing, and online status. Each feature was tested and

improved before moving on to the next, which helped us create a smooth and reliable experience for users.

1.6 Report Organization

On the behalf of the completeness of our project, we have reported some of the goals and our final documented report has been systemized under the following chapters:

Chapter 1: Introduction

This chapter introduces the main idea behind the real-time chat application. It explains the communication problems it aims to solve and outlines the project's objectives. It also includes the scope and limitations of the system.

Chapter 2: Background Study and Literature Review

This section focuses on the foundation and research done before starting the project. It discusses how real-time messaging systems work and reviews similar platforms to understand their features and technologies.

Chapter 3: System Analysis and Design

This chapter explains how the system was planned. It includes requirement gathering, system architecture, feasibility study, and the design phase. Diagrams, flowcharts, and the development process model are also provided here.

Chapter 4: Implementation and Testing

This chapter covers how the system was actually built using the MERN stack (MongoDB, Express.js, React, Node.js). It explains the tools, libraries, and frameworks used, as well as the testing techniques applied to ensure the system works correctly.

Chapter 5: Conclusion and Future Recommendation

The final chapter summarizes the results of the project, presents the main conclusion, and suggests possible improvements or features that could be added in the future to enhance the chat system

CHAPTER 2

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

In today's rapidly evolving digital landscape, real-time communication has become a vital component of both personal interactions and professional collaborations. The widespread adoption of remote work, online education, and virtual social engagement has significantly increased the demand for instant messaging platforms that enable users to stay connected seamlessly. Whether it's coordinating with teammates across different time zones, participating in live classroom discussions, or simply staying in touch with friends and family, users expect communication tools to be fast, reliable, and secure.

However, many of the existing platforms still struggle with key challenges such as poor responsiveness, lack of real-time synchronization, scalability bottlenecks, and insufficient security protocols. These shortcomings often lead to message delays, disconnections, and data vulnerabilities—issues that can greatly impact the overall user experience and trust in the platform.

To overcome these limitations, there is a growing need for a robust, user-centric, and secure real-time chat application that facilitates seamless communication across various use cases. An ideal solution should offer low-latency message delivery, support for multi-user and group conversations, cross-device synchronization, and an intuitive, accessible interface. Additionally, strong data protection measures such as end-to-end encryption, secure authentication, and reliable server architecture are crucial to ensure user privacy and system integrity. By addressing these needs, such a platform can significantly enhance communication efficiency and user satisfaction in both casual and professional settings.

2.2 Literature Review

User Experience (UX) and Interface Design are central to the success of any communication platform. As noted by the Nielsen Norman Group (2020), users are more likely to engage with applications that offer intuitive navigation, clear chat interfaces, and responsive performance across devices. A chat application must allow users to seamlessly create conversations, send messages, and receive real-time notifications with minimal complexity.

Security and Privacy remain top concerns in messaging platforms. With increasing cyber threats, users demand end-to-end encryption, secure user authentication, and data privacy guarantees. Research by Symantec (2021) shows that platforms incorporating encrypted data transfer, role-based access control, and secure login systems (e.g., two-factor authentication or token-based mechanisms) significantly improve user trust and adoption.

Scalability and Performance are also critical. A real-time application must maintain high performance under concurrent user loads and support features such as online presence detection, chat history storage, and group messaging. Technologies like WebSockets (e.g., Socket.IO) and NoSQL databases (e.g., MongoDB) are commonly used to achieve these capabilities by providing efficient data streaming and storage models.

Cross-Platform Compatibility is another important consideration. With users accessing chat platforms from various devices—desktops, laptops, smartphones—applications must be responsive and fully functional across all screen sizes and operating systems. This requires careful front-end development, often using frameworks like React.js, which supports dynamic rendering and reactivity.

In the context of Nepal, where internet usage and smartphone penetration are steadily rising, the relevance of real-time communication tools is becoming increasingly apparent. With the growing trend of remote work and digital collaboration, there is an urgent need for domestic platforms that can serve both local and international communication requirements. However, challenges such as inconsistent connectivity, language preferences, and device limitations must be addressed to ensure widespread adoption.

In conclusion, this literature review emphasizes the key components for a successful real-time chat application: user-friendly design, strong security, high scalability, and reliable performance. By following these best practices and tailoring them to specific needs, the proposed application can effectively meet modern communication

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

The Real-Time Chat Application is a web-based system designed to make online communication faster, easier, and more interactive for users. It provides a centralized platform where individuals can register, log in, and engage in instant messaging through one-on-one chats. Users can send text messages, share media files, view typing indicators, and see online/offline status in real time. The system also supports secure login and data handling to ensure user privacy.

For administrators, the platform includes tools to manage users, monitor chat activity, and maintain system performance. It offers features like real-time message monitoring, deleting functionality, and control over content moderation. By replacing traditional communication methods with a responsive, real-time solution, the chat application ensures smooth message delivery, instant notifications, and secure data storage using modern technologies in the MERN stack.

3.1.1 Requirement Analysis

Functional Requirements

These requirements define the core functionalities of the Real-Time Chat Application, focusing on the actions that users and administrators can perform. They describe the system's expected behavior based on user interactions and are easily quantifiable.

The system must provide the following functionalities:

I. **User Authentication:** Users must be able to log in securely using a username and password. Admins should have elevated access to manage user roles and permissions.
II. **Real-Time Messaging:** Users can send and receive messages instantly, with notifications for new messages in real-time. The system must support both one-on-one and group chats.
III. **Message History:** All sent and received messages should be stored and accessible within a chat history, allowing users to scroll through and view past conversations.

IV. **Online Presence Indicator:** The application must display the real-time online status of users (online/offline) to facilitate communication.

V. **User Management (Admin Functionality):** Only admins have access to manage user accounts, including the ability to block/unblock users, modify user roles, and monitor overall system activity.

VI. **Message Notifications:** Users should receive immediate notifications for new messages or when they are mentioned in a group chat, ensuring timely engagement.

VII. **Profile Management:** Users can update their profile information (such as name, avatar, and status) directly from the platform.

VIII. **Search and Add Contacts:** Users can search for and add other users to their contact list by username or email, facilitating easy connection.

IX. **Security Measures:** The application must employ encryption (e.g., end-to-end encryption for messages) and ensure secure authentication processes to safeguard user data.

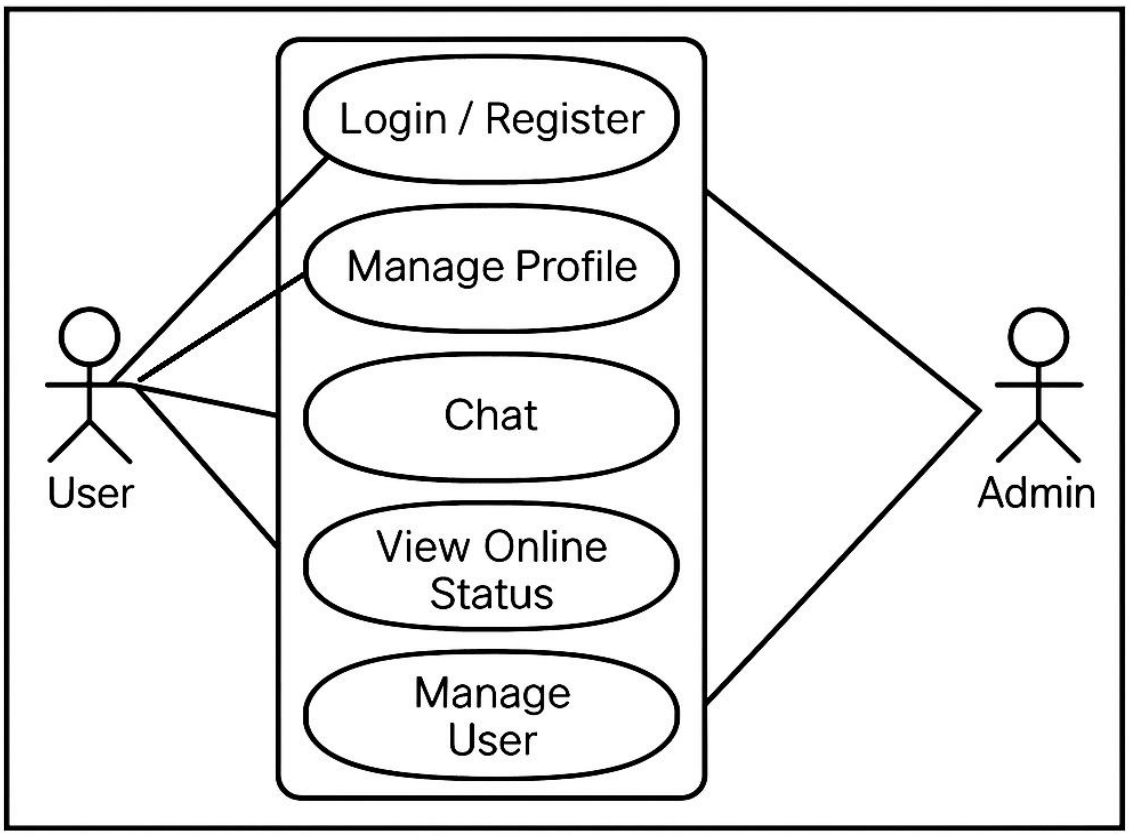


Fig 3.1 Use Case Diagram

Non - Functional Requirement

Usability:

The Real-Time Chat Application must be intuitive and easy to navigate for both technical and non-technical users. Users should be able to send and receive messages, manage contacts, and engage in group chats without any difficulties. The design should ensure that all features are easily accessible, with a clean and straightforward interface. The application should provide an engaging and efficient experience, especially during real-time conversations.

Security:

Security is a top priority for the chat application. It must incorporate robust measures, including **end-to-end encryption** for message exchanges, **secure user authentication** (using multi-factor authentication if necessary), and data protection protocols. The application should be resilient to common security threats like hacking, data leakage, and unauthorized access to private messages.

Performance:

The application should exhibit high performance, ensuring that messages are delivered with minimal latency, even during peak usage times. **Real-time notifications** should be prompt and efficient, with the system responding quickly to user interactions, such as sending messages, joining chat rooms, or updating presence status. Overall, the user experience should be smooth, with minimal delays between message send/receive actions.

3.1.2 Feasibility Analysis

A feasibility study helps determine whether building the real-time chat application is practical and worth pursuing, without actually developing the full system first. It looks at whether the project is technically possible, cost-effective, and aligned with the goals of the organization. The following points highlight the feasibility of this project:

i. Technical Feasibility

The project is technically feasible as it uses widely adopted and modern technologies such as **MongoDB** for the database, **Express.js** and **Node.js** for the backend, and **React.js** for the frontend. Real-time communication will be handled using **Socket.IO**. Development will take place in free environments like **Visual Studio Code**, and deployment can be done on cloud

platforms like **Render**, **Vercel**, or **Heroku**. All tools are well-supported and do not pose any technical limitations

ii. Economic Feasibility

The project will be developed using free and open-source tools and technologies. Since no expensive hardware or paid software is required, the cost of development is minimal. This makes the project economically feasible with no significant financial risks.

ii. Operational Feasibility

The system is designed to be simple and easy to use for both users and administrators. Users can chat, manage their profiles, and view online/offline statuses, while admins can manage user data. All information is stored securely in a cloud-based database, ensuring that data can be accessed and monitored as needed. The system structure supports efficient communication and smooth day-to-day operation.

iv. Schedule Feasibility

This includes the project schedule and all time allocated for their completion. The Gantt chart is as follow:

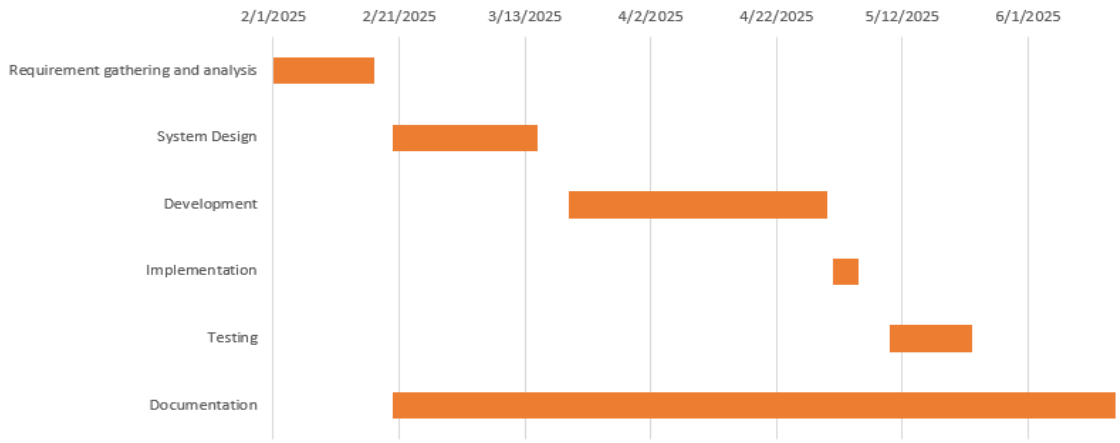


Fig 3.2 Project Gantt Chart

3.1.3 Object Modelling: Class and Object Diagram

Class Diagram

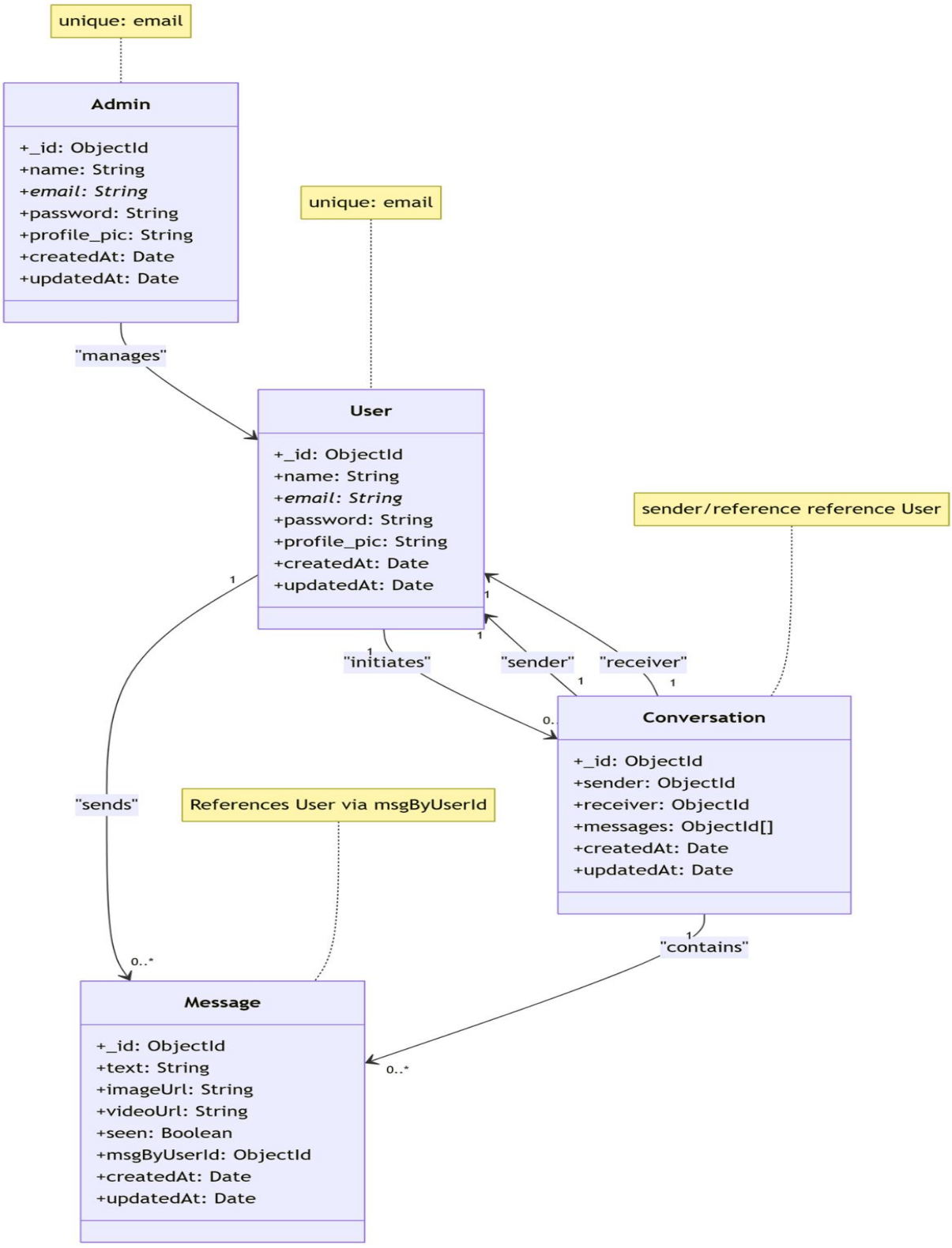


Fig 3.3 Class Diagram

Object Diagram

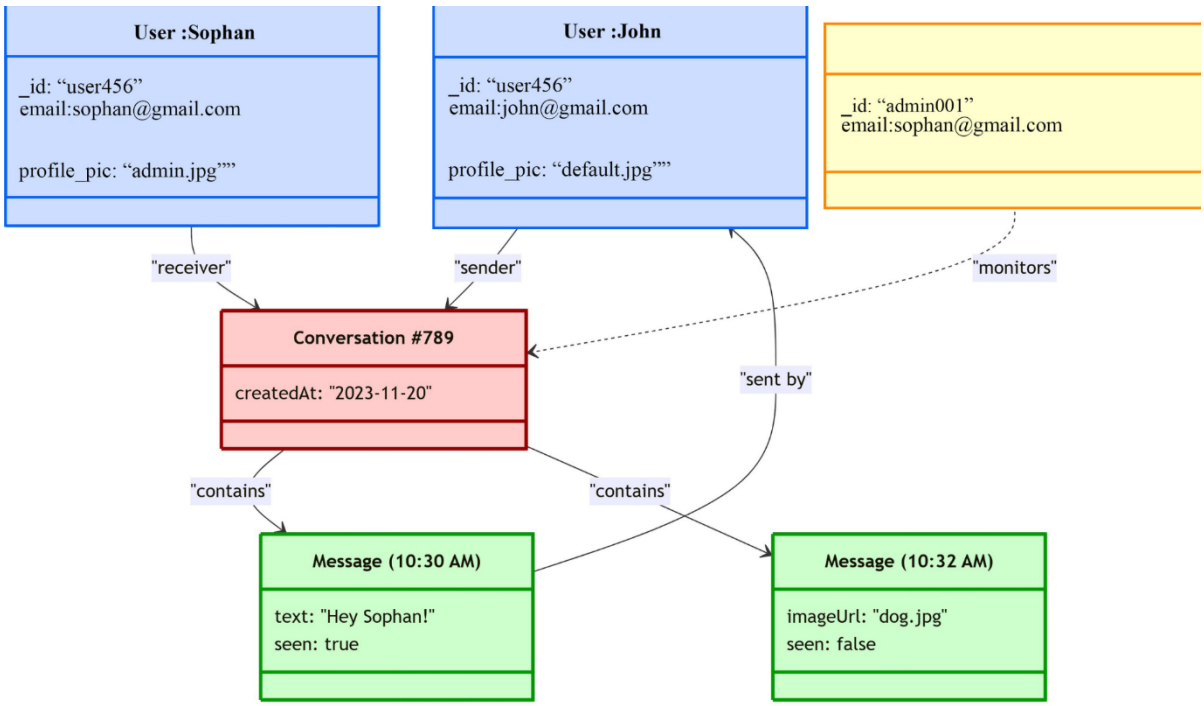


Fig 3.4 Object Diagram

3.1.4 Dynamic Modelling: State and Sequence diagram

State Diagram

State Diagram visualize how an object's change in response to events and conditions, allowing designers and developers to understand and represent complex behavior logic in a clear and organized manner.

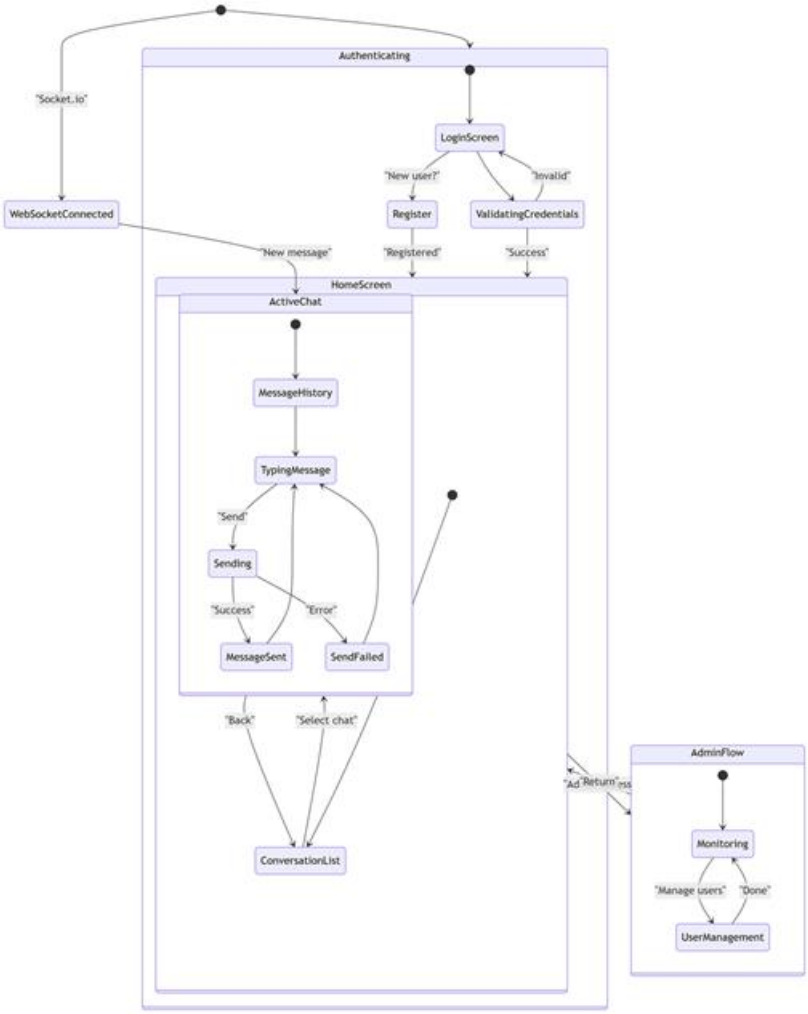


Fig 3.5 State Diagram

Sequence Diagram

Fig 3.5 State Diagram

Sequence Diagram are interaction diagrams that detail how operations are carried out. It describes interactions among classes in terms of an exchange of message over time.

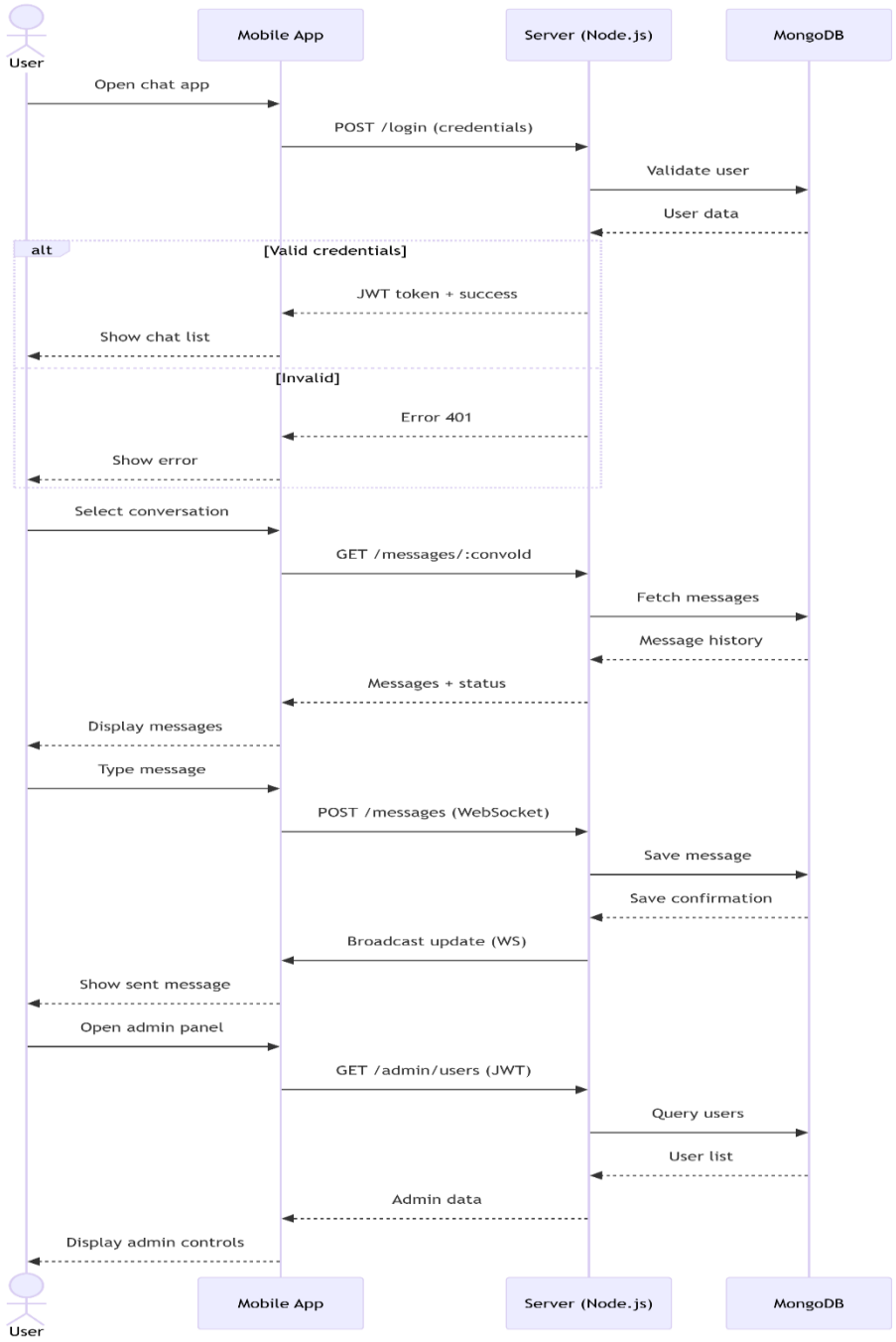


Fig 3.6 Sequence Diagram

3.1.5 Process Modelling: Activity Diagram

The activity diagram is a flowchart to represent the flow of control among the activities in the system. It is another important diagram in UML to describe the dynamic aspects of the system. The activity can be described as an operation of the system.

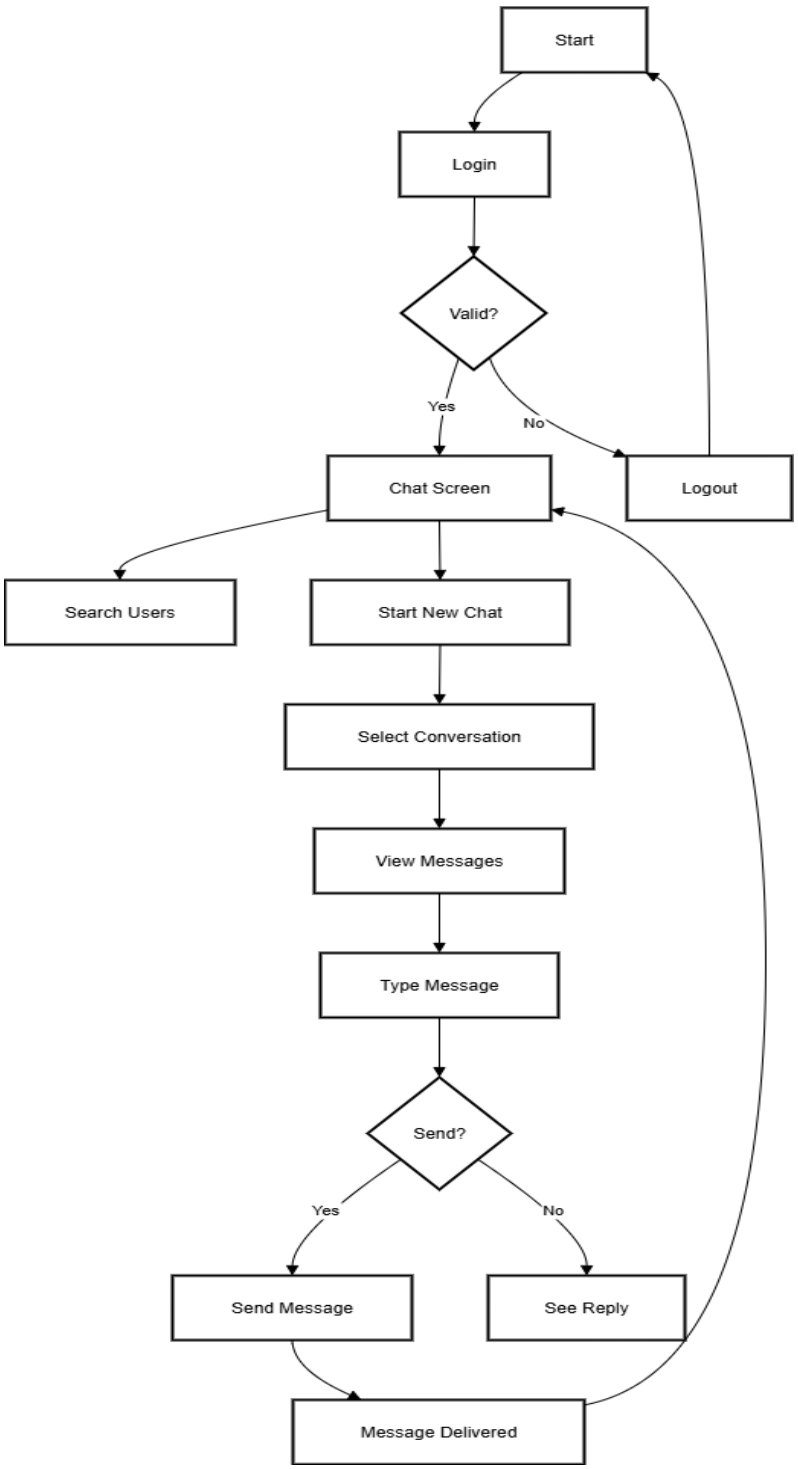


Fig 3.7 Activity Diagram

3.2 System Design

3.2.1 Component Diagram

Component diagram can be described as a special kind of diagram in UML as it shows static implementation view of a system. They are used to visualize the organization and relationships among components in a system. The purpose of the component diagram is to construct executable by using forward and reverse modelling.

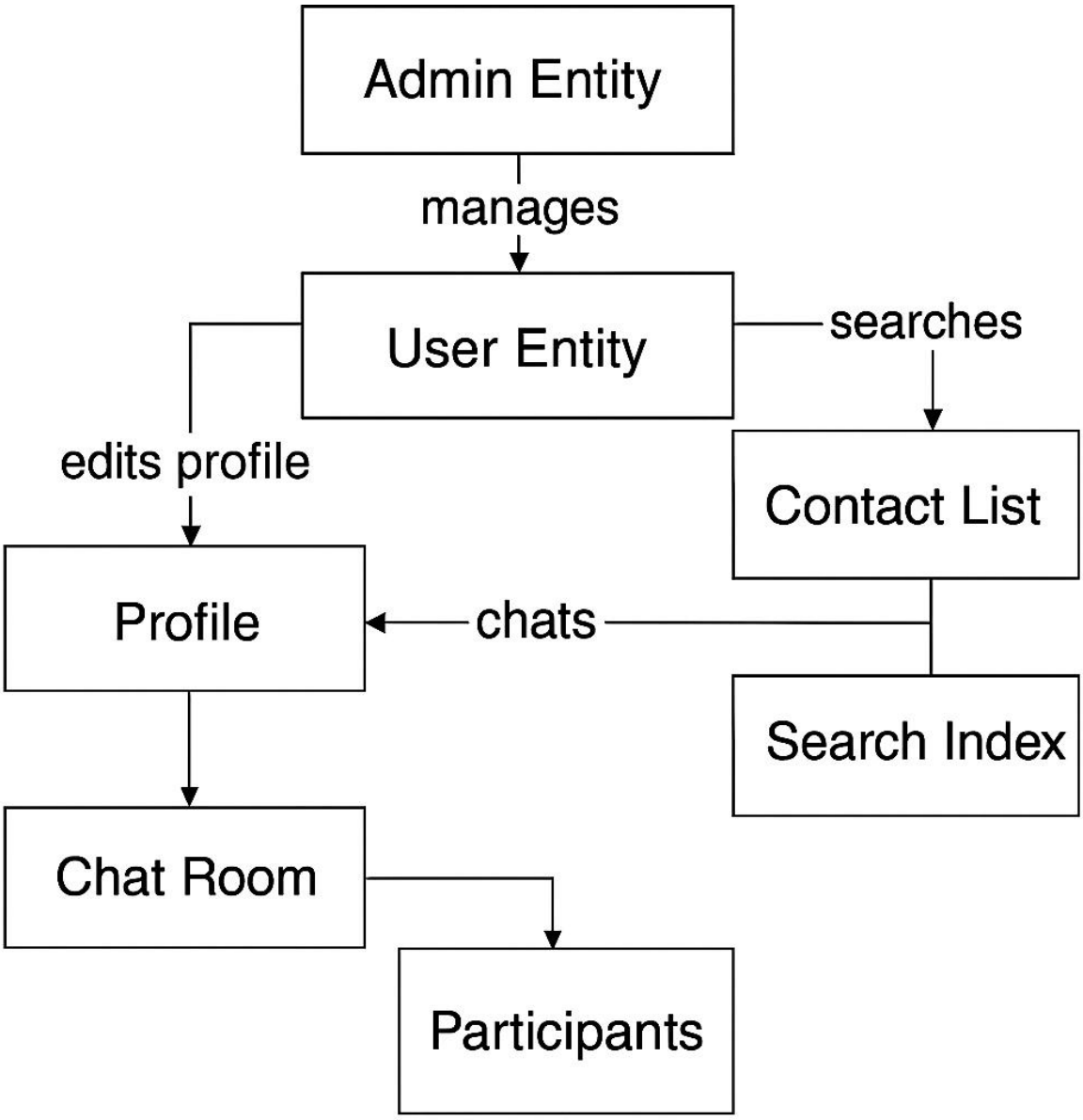


Fig 3.8 Component Diagram

3.2.2 Deployment Diagram

A deployment diagram is a diagram that shows the execution architecture of a system, including nodes such as hardware or software execution environments, and the middleware connecting them. Deployment diagrams are typically used to visualize the physical hardware and software of a system. It consists of nodes and their relationships.

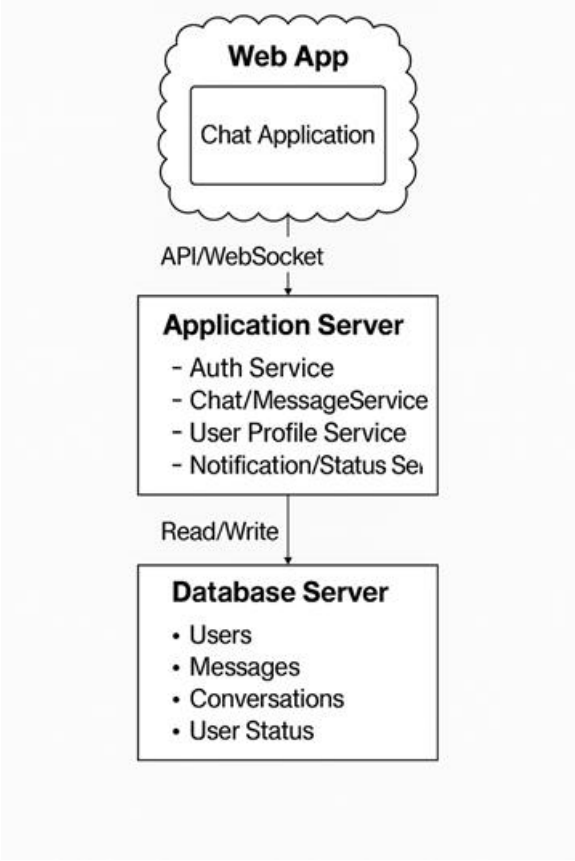


Fig 3.9 Deployment Diagram

Web App

The **Web App** is the frontend of the Chat Application that users access through their browser. It handles user actions like registration, login, sending and receiving messages, viewing online users, joining chat rooms, and editing profiles. It interacts with the backend using REST APIs and WebSockets for real-time communication.

Application Server

The **Application Server** is built using Node.js and Express.js, and it handles all backend services of the chat system. These services include:

I. **Admin Service:** Allows the admin to manage users, monitor conversations, and control access.
II. **User Service:** Handles user registration, login, profile updates, and authentication.
III. **Chat Service:** Manages real-time messaging, chat room creation, and message storage.

Database Server

The **Database Server**, powered by MongoDB, stores all important data used in the chat system. This includes user accounts, chat messages, chat room details, media shared, and login history. The database supports read and write operations from the application server to ensure consistency and real-time updates across the system.

3.2.3 Database Schema Design

The figure below shows the **database schema design** for the Chat Application. A schema defines the structure of data stored in the system. In this chat system, there are multiple collections (similar to tables in SQL databases), such as Users, Messages, Converstation, and Messages. Each collection contains fields (like name, message text, timestamps, etc.) with specific data types. This design helps organize how data is stored, accessed, and related within the application.

Collection 3.1 User and Admin Collection

```
const UserSchema = new mongoose.Schema({
    name :{
        type:String,
        required:[true,"provide name"]
    },
    email:{
        type:String,
        required:[true,"provide email"],
        unique:true
    },
    password :{
        type:String,
        required:[true,"provide password"]
    },
    profile_pic :{
        type:String,
        default:""

    }

},{
    timestamps:true
})
```

Collection 3.2 Conversation Collection

```
const conversationSchema = new mongoose.Schema(
  {
    sender: {
      type: mongoose.Schema.ObjectId,
      ref: "User",
      required: true,
    },
    receiver: {
      type: mongoose.Schema.ObjectId,
      ref: "User",
      required: true,
    },
    messages: [
      {
        type: mongoose.Schema.ObjectId,
        ref: "Message",l
      },
    ],
  },
  {
    timestamps: true ; }}
```

Collection 3.3 Messages Collection

```
const messagesSchema = new mongoose.Schema(
  {
    text: {
      type: String,
      default: "",
    },
    imageUrl: {
      type: String,
      default: "",
    },
    videoUrl: {
      type: String,
      default: "",
    },
    seen: {
      type: Boolean,
      default: false,
    },
    msgByUserId:{
      type: mongoose.Schema.ObjectId,
      ref: "User",
      required: true,
    }
  },
  {
    timestamps: true,
```

3.3 Algorithm Used

3.31 Binary Search :

```
export function binarySearch(users, key, searchValue) {
    let low = 0;
    let high = users.length - 1;
    searchValue = searchValue.toLowerCase();

    while (low <= high) {
        const mid = Math.floor((low + high) / 2);
        const currentValue = users[mid][key].toLowerCase();

        if (currentValue === searchValue) {
            return users[mid]; // Found
        } else if (currentValue < searchValue) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return null; // Not found
}
```

CHAPTER 4

IMPLEMENTATION AND TESTING

4.1 Implementation

In this phase, the theoretical design of the chat application is turned into a working system, utilizing different programming languages and tools to develop both the front end and back end. The tools used for this system are as follows:

4.1.1 Tools Used

Tools that are used while designing this website are

Front end tools

I. **HTML**

HTML is used to structure the chat application pages. It is responsible for creating sections, messages, links, and dynamic components like the chat interface, user lists, etc.

II. **CSS**

CSS is used for styling the front end of the chat application. It is used to design different tags, buttons, and components, making the app responsive and visually appealing. We use a combination of inline, internal, and external CSS

III. **JavaScript**

JavaScript is used to provide client-side validation, enhance interactivity, and add dynamic features. It powers real-time message updates, user typing notifications, and other interactions

IV. **React.js**

React is used to build the user interface. It is ideal for creating dynamic, real-time updates within the chat application, enabling features like new messages appearing without refreshing the page, rendering chat components, and handling user state

V. **React Router**

React Router is used for managing navigation and routing within the chat application. It allows users to move between different views, like the login page, chat rooms, and profile, without reloading the page.

VI. **Redux**

Redux is used to manage the global state of the application, such as user authentication, active chat rooms, and messages. It simplifies state management across various components, ensuring smooth data flow throughout the application.

VII. **TailWind CSS**

Tailwind CSS is a utility-first CSS framework used for styling the chat application quickly and efficiently. It allows developers to apply pre-defined classes directly in the HTML or JSX code, making it easy to build responsive and modern designs without writing custom CSS. Tailwind helps maintain design consistency and speeds up the development process.

Back end tools

I. **Node.js**

Node.js serves as the backend for the chat application. It is responsible for handling the server-side logic, managing user connections, message handling, and real-time communication.

II. **Express.js**

Express is used for building RESTful APIs and routing in the chat application. It simplifies the creation of routes for user login, registration, and message handling.

III. **Socket.io**

Socket.io is used to establish real-time, bi-directional communication between the client and server. It enables real-time messaging features, such as sending and receiving messages instantly.

Database

I. **MongoDB** MongoDB is used for storing user data, messages, chat history, and other necessary information. It is ideal for this application as it is scalable and handles large volumes of data efficiently.

Documentation Tools

I. **MS office**

This is used for writing and editing the documentation about our chat application

II. **Draw.io**

This is used to generate diagrams for the system analysis and design of Chat Application.. Diagrams were created using this tool in order to save time since all components are available with drags and drop functions.

4.1.2 Implementation Details of Modules

Different modules of this system are described as below:

i. **Admin Module:**

Admin can manage the user details like who are connected and delete the user according to there last active status .

User Module

I. **User Registration/Login**: Users can register or log in using credentials (username/password). After successful login, users are redirected to the main chat interface.

II. **Chat Dashboard**: After logging in, users can view the list of available chat rooms, active chats, and other available features like creating new rooms or joining existing ones.

ii. Real Time message

I. **Socket.io** is used to enable real-time, bi-directional communication between the client and server. Users can send and receive messages instantly without the need for refreshing the page.

II. **Message Display**: Messages are dynamically displayed in the chat window as they are received.

III. **Typing Indicators**: Users can see when other users are typing in real-time.

iii. **File upload and share**

I. Users can share files (images, videos, documents) in chat rooms.
II. Files are uploaded to a cloud storage service (such as AWS S3) and are shared as downloadable links within the chat.

4.2 Testing

System testing is done by feeding different datasets and performing rigorous testing to ensure the chat application is functioning as expected.

4.2.1 Testing Cases for Unit Testing

Unit testing is conducted in a modular fashion, where each module (such as message sending, receiving, and user authentication) is tested independently. The following are some of the test cases:

I. **Verify User Authentication**
   a. Test that users can register and log in with valid credentials.
   b. Check that the application prevents registration with duplicate usernames.
II. **Message Sending/Receiving**
   a. Ensure that messages are sent and displayed in real-time in the chat window.
   b. Test that the system displays messages correctly (text, images, files).
III. **Real-Time Updates**
   a. Test that all users in the same chat room receive new messages instantly without needing to refresh the page.
IV. **User Input Validation**
   a. Verify that invalid inputs (e.g., special characters in usernames or messages) are handled appropriately.
   b. Ensure that the system rejects empty messages or messages with restricted content.
V. **Message History**
   a. Verify that the message history is stored and retrieved correctly from MongoDB.

b. Test that the chat history is available when the user logs back in.

VI. **File Upload Validation**

a. Ensure that the system only accepts valid file types (images, PDFs, etc.) and properly handles large files.

Postman API Testing

Postman is used to test the RESTful APIs for user authentication, message handling, and CRUD operations on user data. API endpoints are tested to verify that they work as expected before being integrated into the application. This includes:

i. Testing user registration and login.
ii. Sending and retrieving messages.
iii. Creating and joining chat rooms

4.2.1 Testing cases for Unit Testing

In unit testing, we designed the entire system in modularized pattern and each module is tested. Until we get the accurate output from the individual module, we work on the same module. The input forms are tested so that they do not accept invalid input.

Table 4.1 Test case for user registration

S.No	Test Name	Input	Expected output	Test Result
1.	Open Application	http://localhost:5173/register	Chat application Register Page	Pass
2.	Enter invalid username, email, photo, password, and click signup button	Username= Sophan Email =”” Password = “” Photo = “”	Fill out Password and Email filed	Pass
3.	Enter valid username, email, photo, password, and click signup button	Username = Sophan Email = Sophansstha23@gmail.com Photo =”xyz/jsp” Password =Sophan44444	Registered successful	Pass

Table 4.2 Test case for user login email

S.No	Test Name	Input	Expected output	Test Result
1.	Open Application	http://localhost:5173/	Chat application check email page	Pass
2.	Enter invalid email	Email = diwash@gmail.com	User doesn't exits	Pass
3.	Enter valid email	Email= mishraprajwol23@gmail.com	User exists And redirect to check password field	Pass

Table 4.3 Test case for user login password

S.No	Test Name	Input	Expected output	Test Result
1.	Open Application	http://localhost:5173/checkpassword	Check password page	Pass
2.	Enter invalid password	Password = john cena123	Wrong credentials	Pass

3.	Enter valid password	Password = sophan4444	Redirect to home page	Pass

Table 4.4 Test case for admin register

S.No	Test Name	Input	Expected output	Test Result
1.	Open Application	http://localhost:5173/register	Chat application Register Page	Pass
2.	Enter invalid username, email, photo, password, and click signup button	Username= Sophan Email =”” Password = “” Photo = “”	Fill out Password and Email filed	Pass
3.	Enter valid username, email, photo, password, and click signup button	Username = Sophan Email = Sophansstha23@gmail.com Photo =”xyz/jsp” Password =Sophan44444	Registered successful	Pass

Table 4.5 Test case for Admin Login page

S.No	Test Name	Input	Expected output	Test Result
1.	Open Application	http://localhost:5174/	Login page	Pass
2.	Enter the invalid email password	Email: goku@gmail.com Password : 01365	Wrong credentials	Pass
3.	Enter valid Email and password	Emial : “sophanshresth@gmail.com”	Redirect to dashboard	Pass

Table 4.6 Test case for user search

S.No	Test Name	Input	Expected output	Test Result
1.	Open Application	http://localhost:5173/home	Search user page	Pass

2.	Enter invalid name	Name = 12345	User does not exits	Pass
3.	Enter valid user name	Name : sophan	Show user profile	Pass

Table 4.7 Test case for chat

S.No	Test Name	Input	Expected output	Test Result
1.	Open Application	http://localhost:5173/home/67dea806c2bfe5d8dc953dec	Chat board open	Pass
2.	Send a message or video/images	Text = hi how are you	Message send	Pass

Table 4.8 Test case for edit user profile

S.No	Test Name	Input	Expected output	Test Result
1.	Open Application	http://localhost:5173/home/67dea806c2bfe5d8dc953dec	Open the edit page	Pass
2.	Change name and photo as your wise	Name = harry Photo : abc.jpg	Edit profile	Pass

Table 4.9 Test case Admin view user detail

S.No	Test Name	Input	Expected output	Test Result
1.	Open Application	http://localhost:5174/viewuser/67d5225a2b7a3f7ff375a9e4	User detail	Pass
2.	View user Information	Delete user ,see user info,see connected user	See user details	Pass

Table 4.10 Test case to delete user

S.No	Test Name	Input	Expected output	Test Result
1.	Open Application	http://localhost:5174/viewuser/67d5225a2b7a3f7ff375a9e4	User detail Pages	Pass
2.	Click on delete button	Delete use : yes / no option	User deleted succefully	Pass

CHAPTER 5

CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

We have successfully built a Chat Application using the MERN stack (MongoDB, Express.js, React, and Node.js). The app allows users to register, log in, and chat with each other in real-time. It has a clean and easy-to-use interface. With the help of Socket.io, messages are sent and received instantly, which makes the chatting experience smooth.

This project is made for small to medium-sized users and doesn't need any special or expensive system to run. Its simple design and real-time features make it a useful and reliable app for anyone who wants to communicate quickly and easily.

5.2 Lesson Learnt

While working on this chat app, we learned how to apply what we know to real-life problems. We improved our technical skills by solving issues like user login, sending and receiving messages in real-time, and keeping data safe and organized.

We also learned how important it is to write clean and organized code so that the project is easy to update and fix later. We practiced using tools like Git for version control and learned how to work better with databases and APIs.

5.3 Future Recommendation

Our Chat Application already includes the main features needed for messaging, but there are several ways we can improve it to make the experience even better for users.

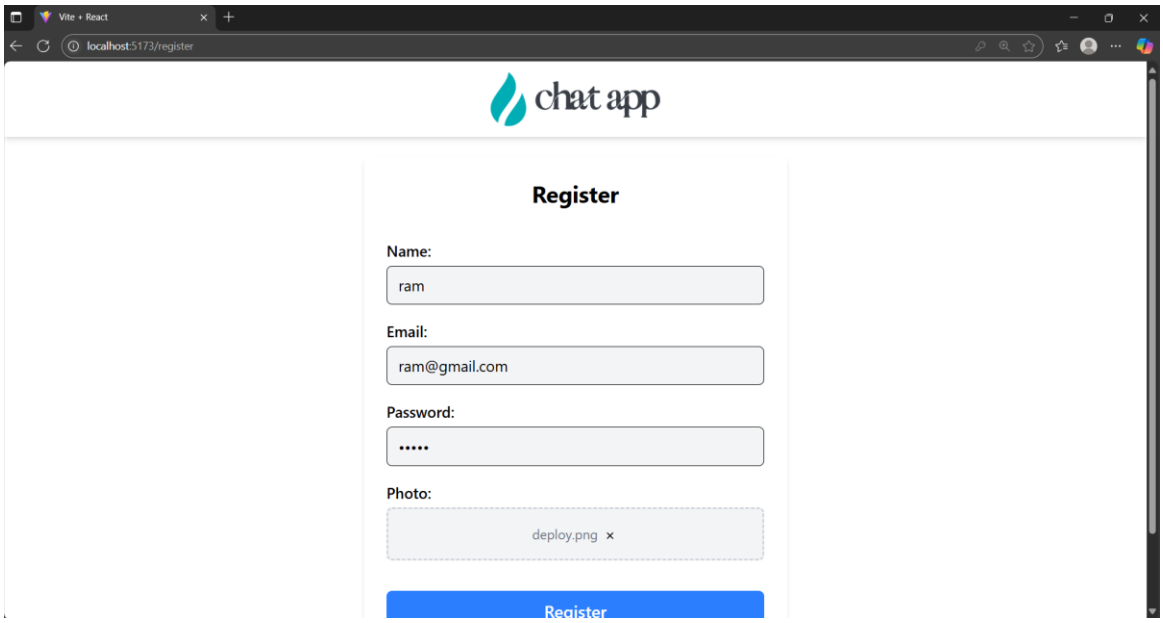
One helpful feature we plan to add is the ability to block users. This will let people block others if they receive unwanted or disturbing messages, making the app safer and more comfortable to use.

We also want to add group chat, so users can create or join group conversations and chat with multiple people at the same time. This will make the app more useful for both friends and teams who want to stay connected.

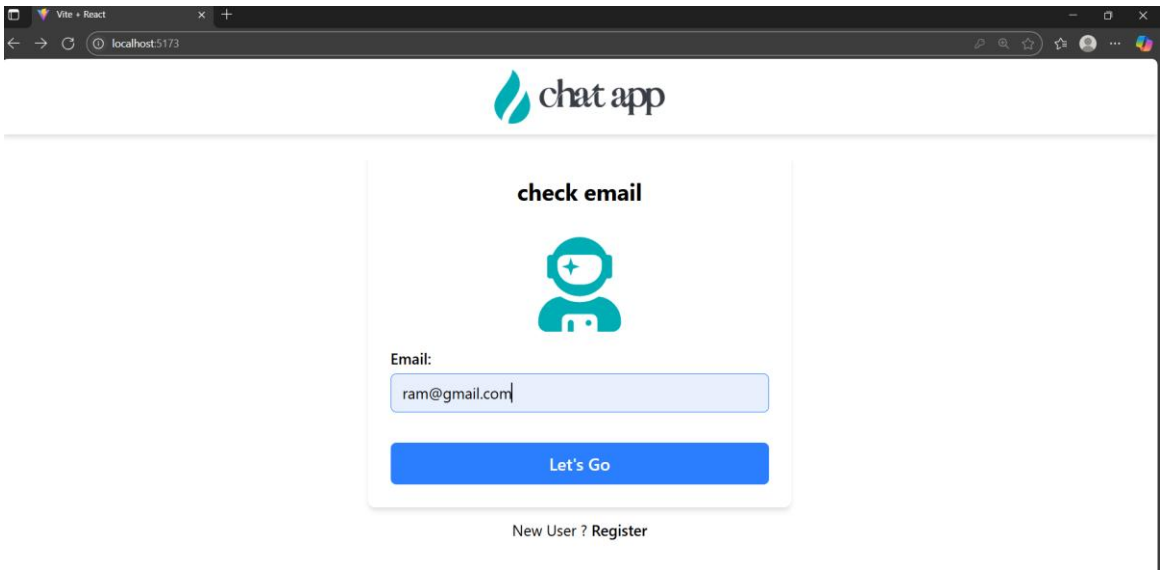
Another exciting idea is to include AI-based chat support. This could be used to help users with common questions, guide new users, or even offer automatic replies. It will make the app smarter and more helpful.

In the future, we may also improve security by adding an OTP (One-Time Password) system during login or signup. This will help protect user accounts from being accessed by others. Lastly, adding chat themes or customization options, as well as status updates, could make the chat experience more personal and fun.

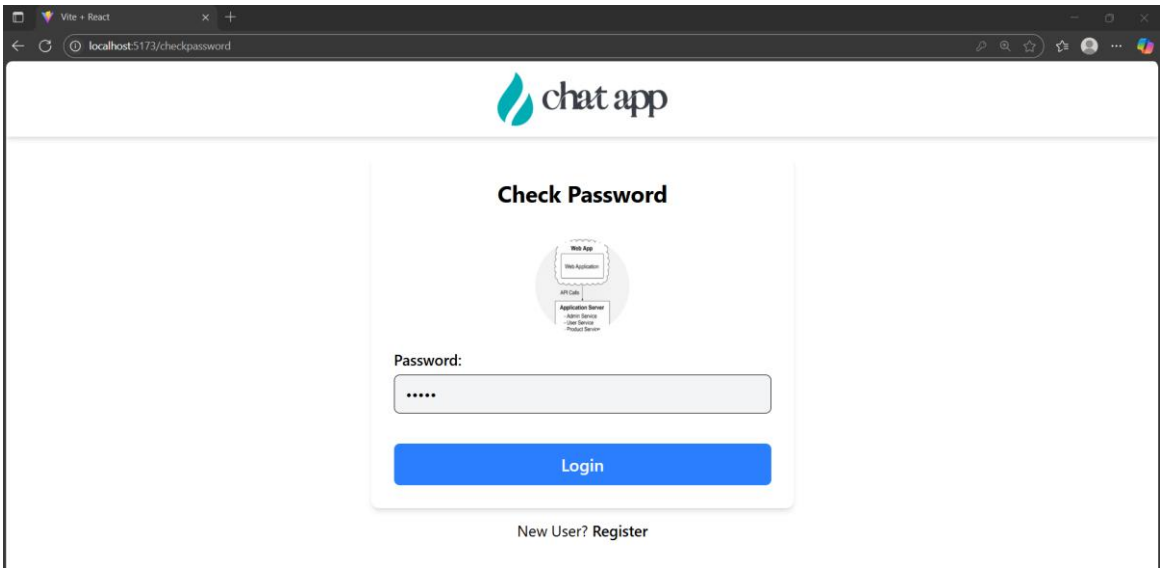
Register Page



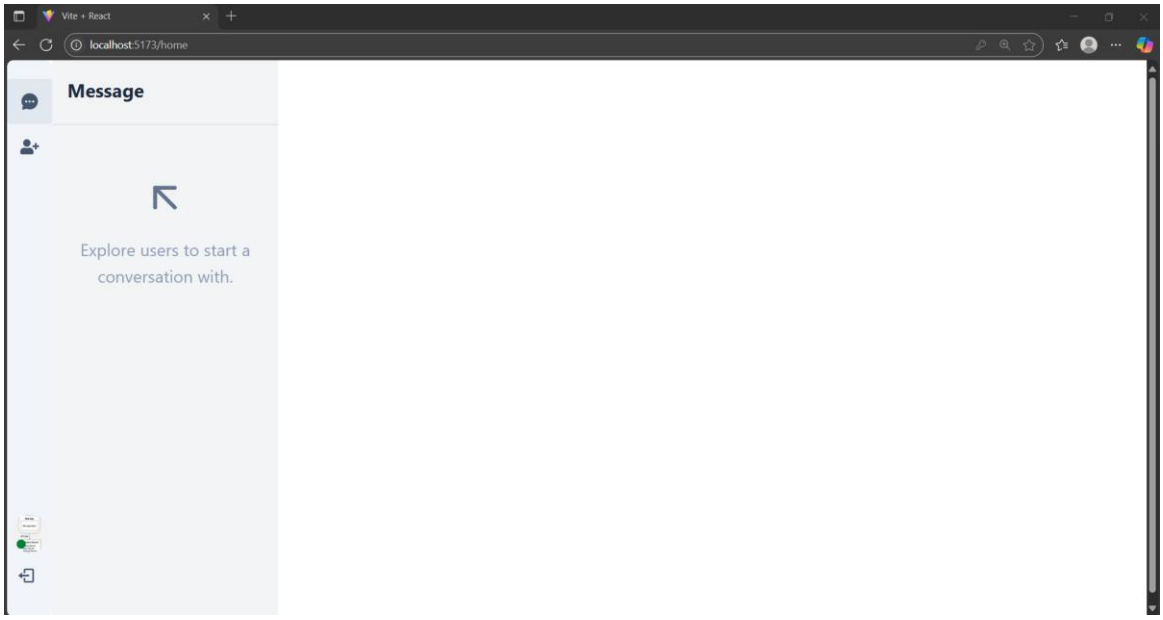
Login email Page



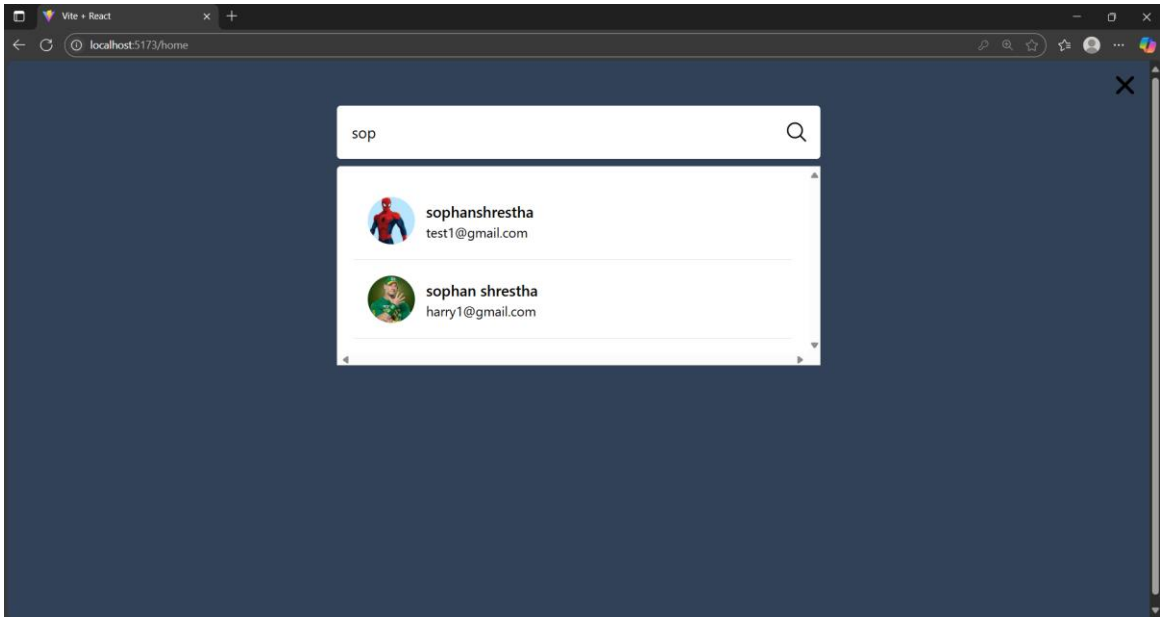
Login password page



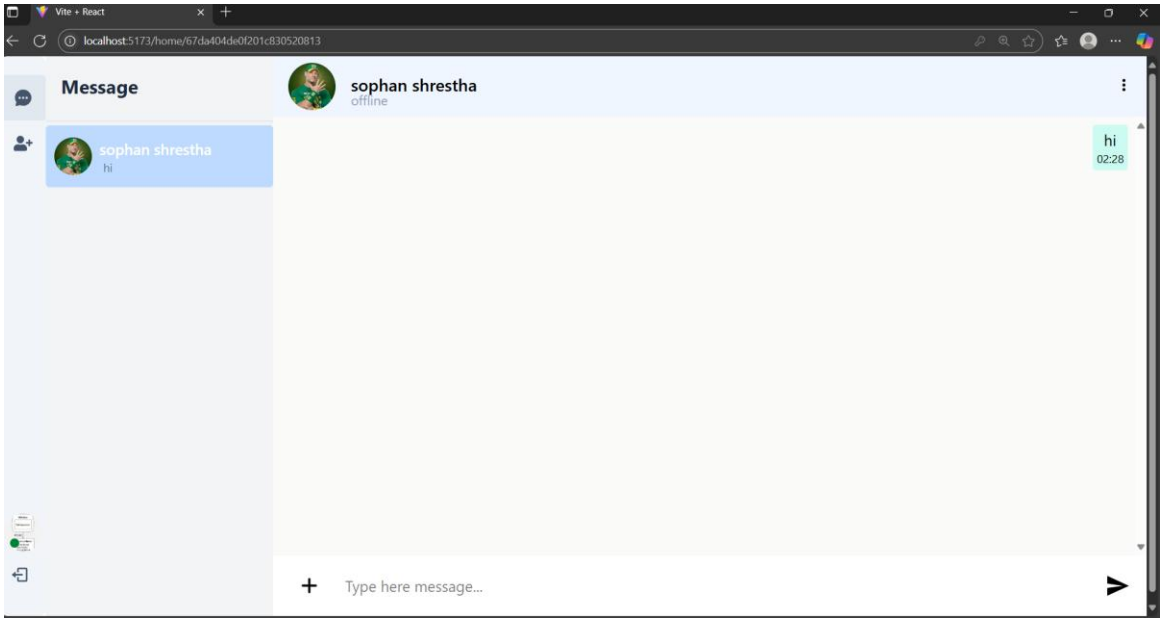
Home Page



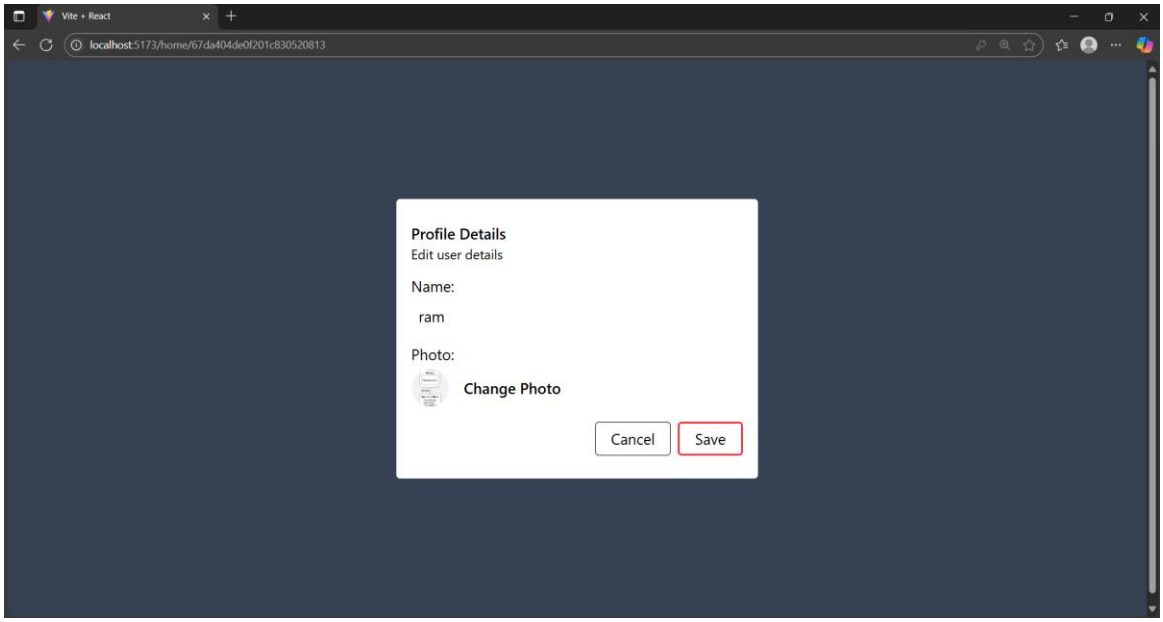
Search Page



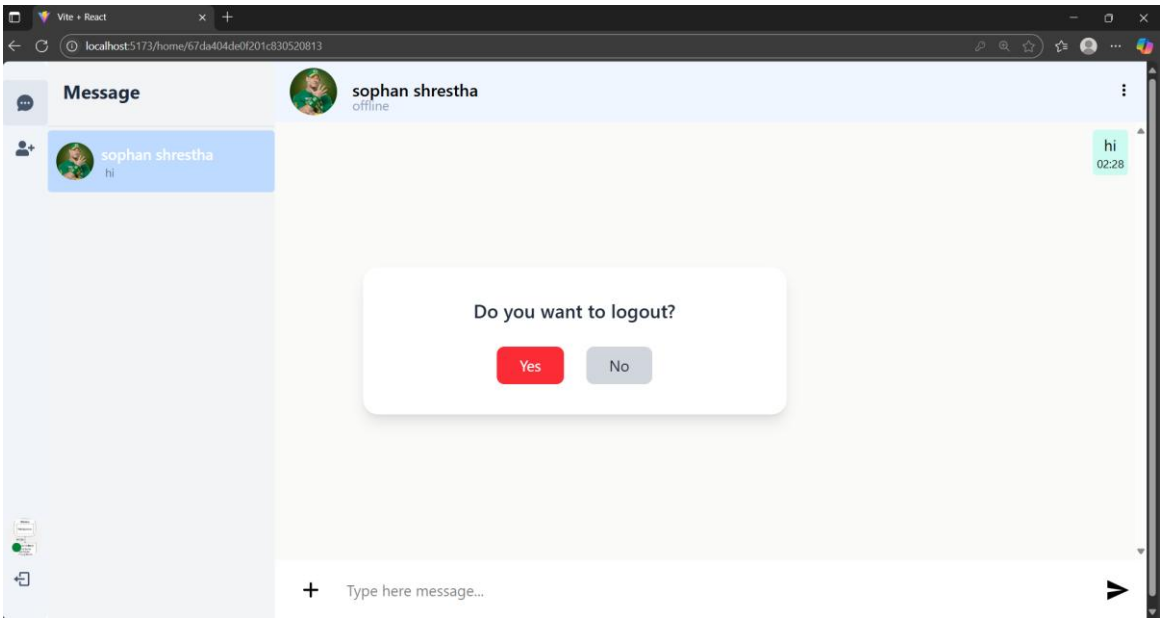
Chat page



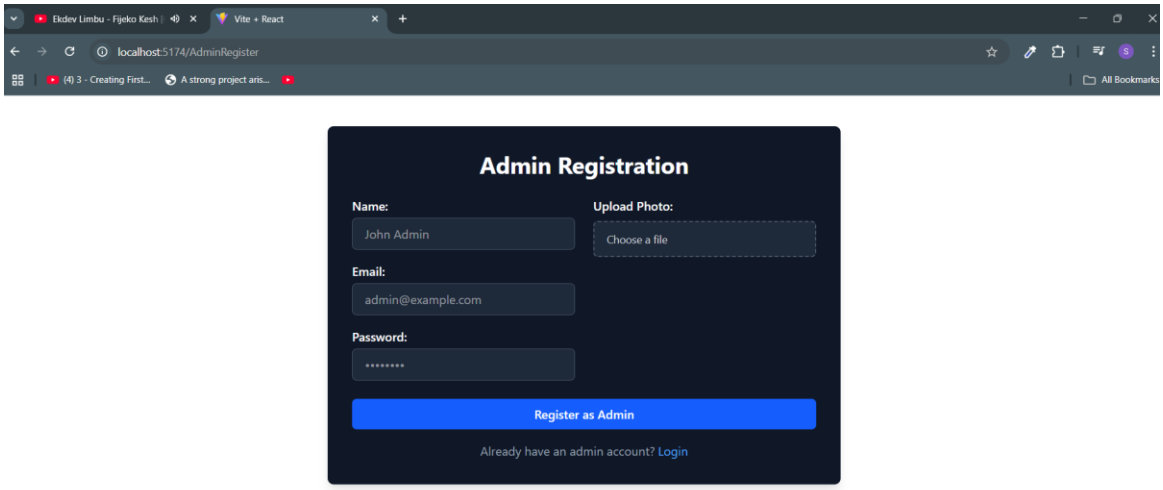
Edit user info Page



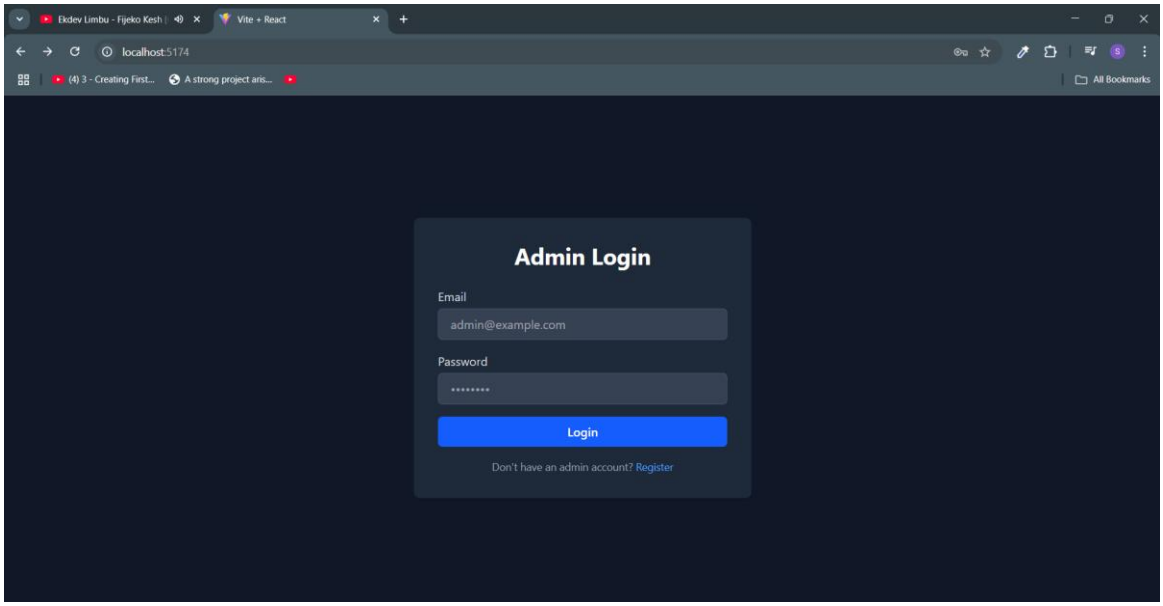
Logout page



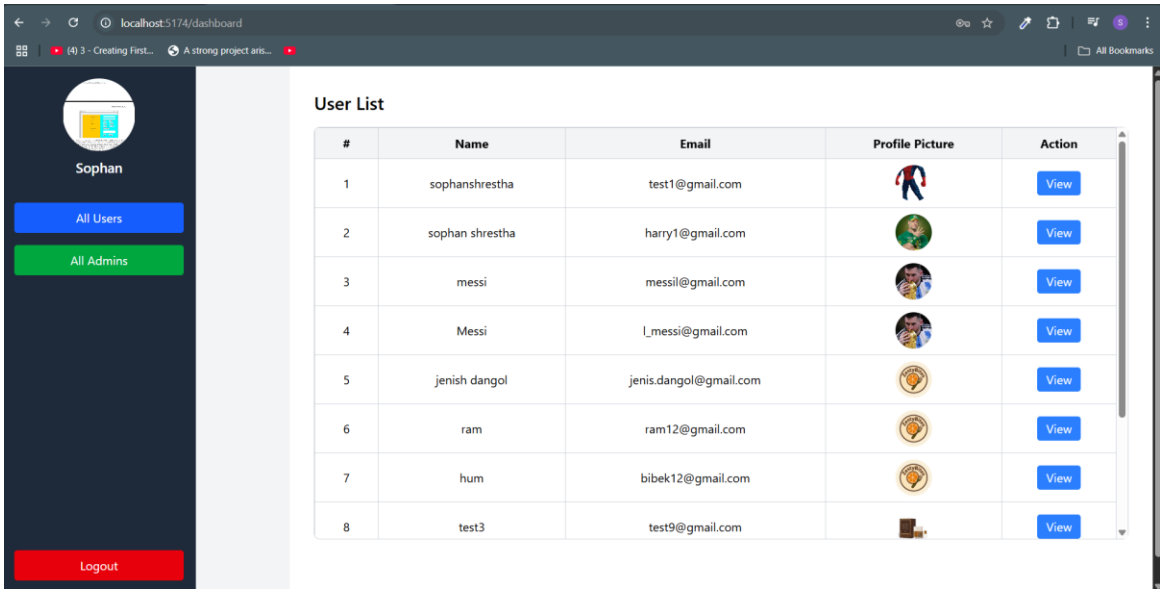
Admin Register Page



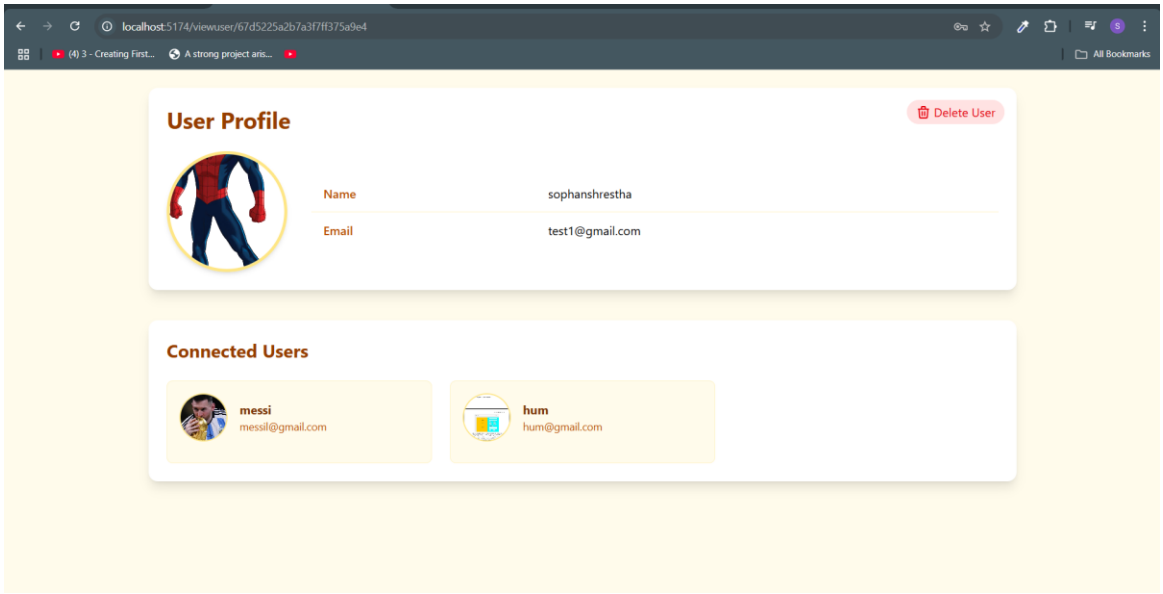
Admin login Page



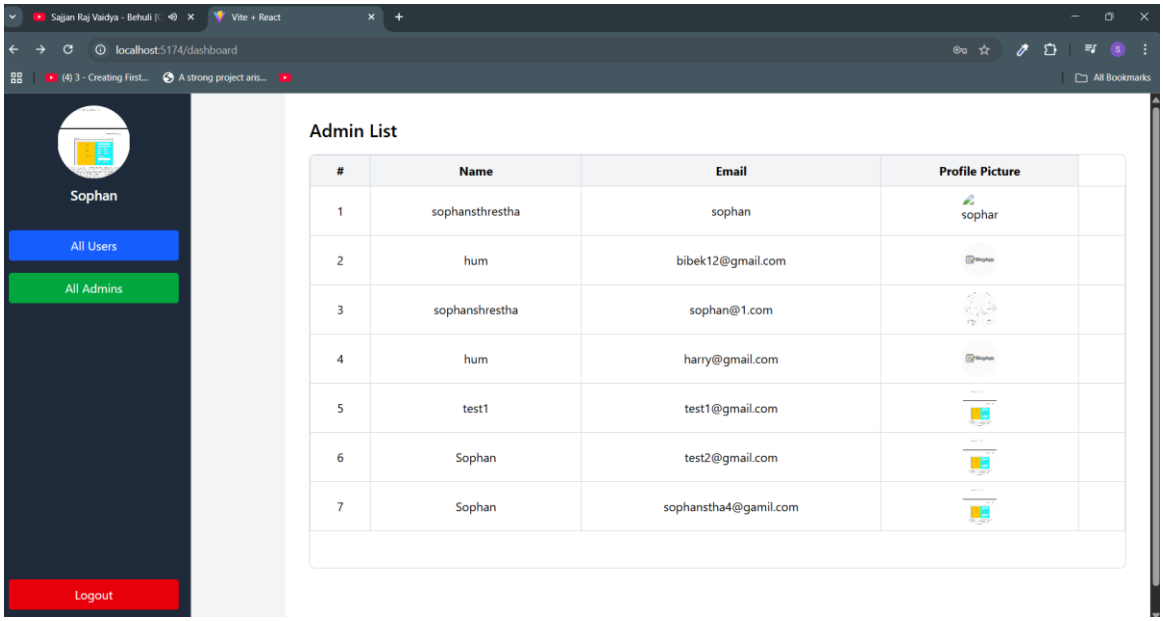
Admin Dashboard Page



All user detail Page



All Admin list Page



References

 K. Beck et al., *Manifesto for Agile Software Development*, 2001. [Online]. Available: https://agilemanifesto.org [Accessed: 27 February 2025].

 Statista, *"Most popular global messaging apps – WhatsApp, Messenger, and Viber user statistics,"* 2024. [Online]. Available: https://www.statista.com/statistics/258749/most-popular-global-mobile-messenger-apps/ [Accessed: 27 February 2025].

 Nielsen Norman Group, *"Chat and Messaging App User Experience,"* 2023. [Online]. Available: https://www.nngroup.com/articles/chat-ux/ [Accessed: 27 February 2025]